

Laboratory of Mobile Operating Systems

File access in Android

Introduction

Android applications can access local files either directly or through a dedicated system picker. Both methods have their own limitations and can be quite confusing. To make today's tasks a little easier, **additional Kotlin files are provided along with this instruction** – feel free to use them however you like.

Direct file access via filepaths

This method can be used to access both the app's private files, as well as public files located in shared storage. The following example shows an easy way of obtaining the path of the application's private directory:

```
val privateAppDirPath = applicationContext.filesDir
```

Paths to shared files can be obtained in various ways – the following example shows a method of finding the correct path of the *Download* directory located in the device internal storage:

```
val downloadDirPath =  
    File(Environment.getExternalStorageDirectory(), "Download")
```

With this knowledge, you should now be able to construct proper filepaths in the Android file system. Sample methods for reading and writing files are provided in the additional *FilesHelper.kt* file. However, be warned that direct access to files located in shared storage requires obtaining a special type of permission from the user. First of all, the following code should be added before the `<application>` node in the *AndroidManifest.xml* file:

```
<uses-permission  
    android:name="android.permission.MANAGE_EXTERNAL_STORAGE"  
    tools:ignore="ScopedStorage" />
```

Then, whenever access to public files is required, the app should verify its permission for managing external storage and request it from the user if needed:

```
if (!Environment.isExternalStorageManager()) {  
    val intent = Intent(  
        ACTION_MANAGE_APP_ALL_FILES_ACCESS_PERMISSION,  
        Uri.parse("package:${BuildConfig.APPLICATION_ID}")  
    )  
    storagePermissionResultLauncher.launch(intent)  
}
```

Finally, here is a template for handling the user's answer to our request:

```
val storagePermissionResultLauncher = registerForActivityResult(
    ActivityResultContracts.StartActivityForResult()) {
    if (Environment.isExternalStorageManager()) {
        // Permission has been granted.
    }
}
```

Indirect file access via URIs

This method uses the Storage Access Framework to browse files with the system picker. While it may be a more elegant solution, it cannot be used for accessing the application's private files. Since communication with the system picker is handled by Intents, a dedicated launcher should be prepared in order to use the obtained URI:

```
val startForResultFileOpen = registerForActivityResult(
    ActivityResultContracts.StartActivityForResult()) {
    result: ActivityResult ->
        val data: Intent? = result.data
        data?.let {
            val uri: Uri? = data.data
            // Use the URI to either read the file or write to it.
        }
}
```

Sample methods for reading and writing files are provided in the additional *ScopedStorageHelper.kt* file.

Task 1. (4 points)

Create a simple text editor, capable of reading text files, displaying and modifying their contents, and saving them as new files.

Variant I

All files should be stored in a public directory, such as the *Downloads* folder.

Variant II

All files should be stored in the application's private directory.

Task 2. (1 points)

Variant I

Update your app with an option of displaying the path or URI of the most recently opened file.

Variant II

Update your app with an option of displaying the list of all text files stored inside your app's private directory.